

UNIVERSIDAD DE OVIEDO

Master en Ingenieria Informatica

Interfaces Multimodales (IMM)

Practica Final

Vision Artificial e Integracion Multimodal

ChefZeroWaste

Sergio Pociño

Sergio Botana

Nicolas Iglesias

17 de mayo de 2026

Índice

1. Composicion del grupo	1
2. Resumen del sistema	1
3. Descripcion de los puntos obligatorios	1
3.1. Entrenamiento de gestos personalizados	1
3.1.1. Diccionario visual	1
3.1.2. Dataset local	2
3.1.3. Captura de datos	3
3.1.4. Transfer Learning con Model Maker	3
3.1.5. Prediccion unimodal visual	3
3.1.6.Codigo involucrado en el punto 1	4
3.2. Integracion con el modulo de PLN	4
3.2.1. Origen del modulo textual	4
3.2.2. Intenciones textuales	5
3.2.3. Complementariedad texto-vision	6
3.2.4. Codigo involucrado en el punto 2	7
3.3. Fusion semantica tardia	7
3.3.1. Integrador en Python	7
3.3.2. Gestion de asincronia	8
3.3.3. Validacion semantica	8
3.3.4. Evidencia de funcionamiento	9
3.3.5. Codigo involucrado en el punto 3	9
4. Descripcion de las ampliaciones	9
4.1. Ampliacion A: Embeddings multimodales Vision + Texto	9
4.1.1. Objetivo	9
4.1.2. Arquitectura de la ampliacion	9
4.1.3. Modelo y espacio compartido	10
4.1.4. Gestos obligatorios conservados	10
4.1.5. Comandos zero-shot	11
4.1.6. Evidencia de funcionamiento de la ampliacion	11
4.1.7. Ejecucion de la ampliacion	12
4.1.8. Codigo involucrado en la ampliacion	12
4.2. Ampliacion B: Integracion de voz en tiempo real	12
4.2.1. Objetivo	12
4.2.2. Arquitectura de la ampliacion	13
4.2.3. Captura de audio y transcripcion	13
4.2.4. Reto de asincronia real	14
4.2.5. Ejecucion	14
4.2.6. Codigo involucrado en la ampliacion	15
4.3. Ampliacion C: Logica de desambiguacion mutua	15
4.3.1. Objetivo	15
4.3.2. Arquitectura de la desambiguacion	15
4.3.3. Umbrales de confianza	16
4.3.4. Tablas de correccion mutua	16
4.3.5. Origen de la confianza en cada canal	17

4.3.6.	Evidencia de funcionamiento	17
4.3.7.	Salida JSON con metadatos de desambiguacion	18
4.3.8.	Codigo involucrado en la ampliacion	19
4.4.	Ampliacion D: Head Tracking como tercer canal de percepcion pasiva . . .	19
4.4.1.	Objetivo	19
4.4.2.	Arquitectura de la ampliacion	20
4.4.3.	Zonas de la pantalla	20
4.4.4.	Estimacion de la pose con solvePnP	20
4.4.5.	Canal gaze pasivo	21
4.4.6.	Evidencia de funcionamiento	21
4.4.7.	Ejecucion	22
4.4.8.	Codigo involucrado en la ampliacion	23
5.	Instrucciones de ejecucion	23
5.1.	Aplicacion obligatoria	23
5.2.	Ampliacion CLIP	24
5.3.	Ampliacion voz ASR	24
5.4.	Ampliacion Head Tracking	24

Resumen

Este documento describe la practica final de Vision Artificial para la asignatura de Interfaces Multimodales. El sistema desarrollado integra un canal visual basado en gestos personalizados, un canal textual de PLN procedente del agente ChefZeroWaste y un integrador de fusion semantica tardia. Se documentan cuatro ampliaciones opcionales: embeddings multimodales con CLIP para emparejamiento zero-shot, integracion de voz en tiempo real con Whisper, logica de desambiguacion mutua que permite a un canal con alta certeza corregir errores del canal contrario, y head tracking como tercer canal de percepcion pasiva que aporta contexto espacial a la fusion.

1. Composicion del grupo

- Sergio Pociño.
- Sergio Botana.
- Nicolas Iglesias.

2. Resumen del sistema

La aplicacion final se basa en el dominio ChefZeroWaste. El objetivo es que el usuario pueda interactuar con un asistente de cocina mediante dos canales complementarios:

- **Canal visual:** reconoce gestos personalizados entrenados con MediaPipe Model Maker.
- **Canal textual:** clasifica intenciones PLN del agente ChefZeroWaste.
- **Integrador:** recibe eventos JSON con intencion y marca temporal, mantiene una ventana de memoria y ejecuta una accion final solo si la combinacion texto-vision es valida.

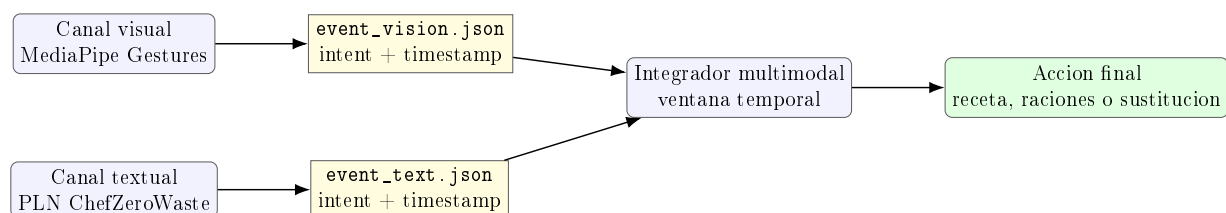


Figura 1: Arquitectura general de la fusion semantica tardia.

3. Descripcion de los puntos obligatorios

3.1. Entrenamiento de gestos personalizados

3.1.1. Diccionario visual

Se ha definido un diccionario visual de tres gestos personalizados nuevos, mas una clase negativa `None`. Los gestos estan adaptados al contexto de ChefZeroWaste:

Clase del modelo	Intencion visual	Uso dentro de la aplicacion
pizca_sal	iniciar_receta	Gesto de “vamos a cocinar”. Confirma el inicio de una recomendacion de receta.
corte_cuchillo	aumentar_raciones	Gesto de partir o cortar. Se usa para adaptar la receta a mas raciones.
sustituir_ingredient	marcar_sustitucion	Gesto de cruz. Marca que se desea sustituir un ingrediente.
None	Sin accion	Clase negativa para separar fondos o poses no intencionales de gestos validos.

La clase **None** no se considera uno de los tres gestos obligatorios, pero mejora la robustez del clasificador al permitir que el modelo aprenda ejemplos donde no debe publicar ningun evento visual.

3.1.2. Dataset local

El dataset local se encuentra en `vision/dataset`. La distribucion final de muestras es:

Clase	Muestras
corte_cuchillo	86
None	82
pizca_sal	119
sustituir_ingredient	125

Cuadro 2: Distribucion de imagenes del dataset de gestos.

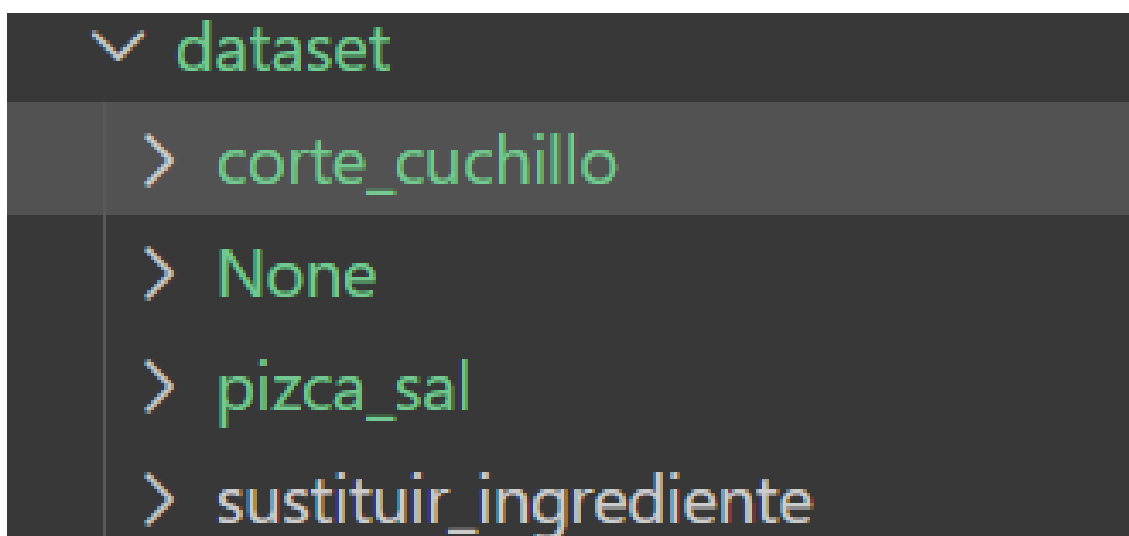


Figura 2: Vista de las carpetas del dataset local con las cuatro clases de gestos.

3.1.3. Captura de datos

La captura de muestras se realizo con el script `vision/captura_datos.py`. Este script abre la webcam, permite indicar la clase que se esta capturando y guarda las imagenes en la carpeta correspondiente del dataset. La estructura final permite que Model Maker cargue automaticamente cada subcarpeta como una clase.

3.1.4. Transfer Learning con Model Maker

El entrenamiento se realizo en Google Colab mediante el cuaderno:

`Entrenamiento_Gestos (1).ipynb`

El cuaderno usa MediaPipe Model Maker para aplicar transfer learning sobre los modelos base de deteccion y representacion de manos. La practica conserva tambien un script equivalente para entrenamiento local:

`vision/entrenar_gestos.py`

El flujo de entrenamiento fue:

1. Cargar el dataset desde carpetas empleando `gesture_recognizer.Dataset.from_folder(...)`.
2. Dividir el dataset en entrenamiento, validacion y test.
3. Configurar hiperparametros con `gesture_recognizer.HParams(...)`.
4. Entrenar la capa final de clasificacion empleando `gesture_recognizer.GestureRecognizer.create_from_model(...)`.
5. Exportar el modelo final con `model.export_model()`.

El modelo final exportado se encuentra en:

`vision/gesture_recognizer.task`

3.1.5. Prediccion unimodal visual

El script visual es `vision/main.py`. Carga el modelo `gesture_recognizer.task`, abre la webcam y publica eventos visuales en `event_vision.json`. Cada evento incluye fuente, intencion visual, gesto crudo, confianza y timestamp.

```
1 {  
2   "source": "vision",  
3   "intent": "iniciar_receta",  
4   "gesture": "pizca_sal",  
5   "timestamp": 1778945740.10,  
6   "confidence": 0.91  
7 }
```

Listing 1: Ejemplo de evento visual publicado por el canal de gestos.

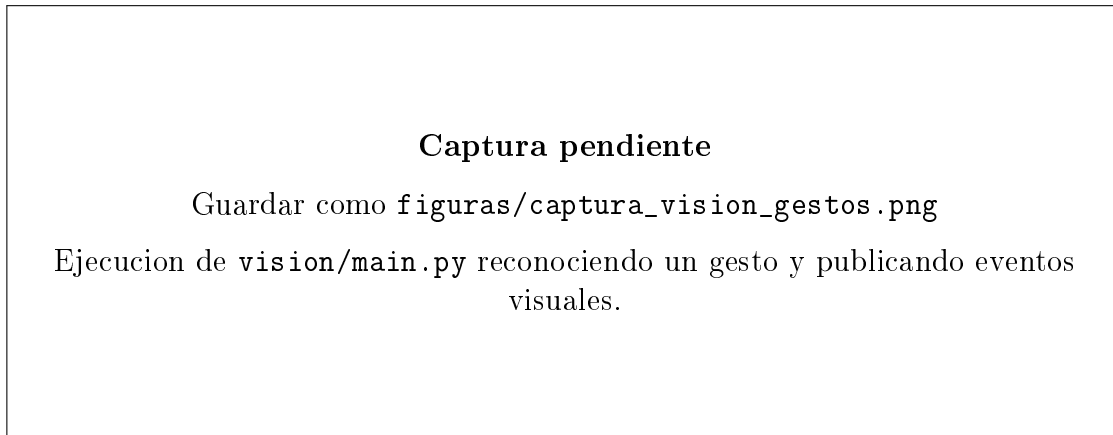


Figura 3: Ejecucion de `vision/main.py` reconociendo un gesto y publicando eventos visuales.

3.1.6. Código involucrado en el punto 1

El código completo está incluido en la entrega. Los ficheros relevantes son:

- `vision/captura_datos.py`: captura muestras desde webcam y las guarda por clase.
- `vision/entrenar_gestos.py`: versión local del entrenamiento con Model Maker.
- `Entrenamiento_Gestos (1).ipynb`: cuaderno usado para el entrenamiento.
- `vision/main.py`: ejecución del canal visual y publicación de eventos.
- `vision/gesture_recognizer.task`: modelo final exportado.

Fragmento clave del canal visual: las clases del modelo se traducen a intenciones semánticas antes de publicar el JSON.

```
1 INTENCIONES_VISUALES = {  
2     "pizca_sal": "iniciar_receta",  
3     "corte_cuchillo": "aumentar_raciones",  
4     "sustituir_ingrediente": "marcar_sustitucion",  
5 }
```

Listing 2: Mapeo semántico del canal visual.

3.2. Integración con el módulo de PLN

3.2.1. Origen del módulo textual

El módulo de PLN parte del bloque anterior de la asignatura y del agente conversacional ChefZeroWaste. En el proyecto final se conserva una arquitectura basada en Bag of Words y clasificación de intenciones. Las prácticas de referencia de PLN de la asignatura son:

- `S1TextClassificationBoW`: clasificación con Bag of Words.
- `S2TextClassificationVectorsSinGloVeVectors`: representaciones vectoriales.
- `S3TextClassificationMLMeasures`: evaluación con métricas de ML.

En la entrega final, el dominio ya no es genérico, sino ChefZeroWaste: ingredientes disponibles, recomendaciones de recetas, raciones, lista de compra y sustituciones.

3.2.2. Intenciones textuales

El canal textual se implementa en `main_chef_zero_waste.py` y `CanalTextoChefZeroWaste.py`. Intenciones clasificadas:

Intencion textual	Descripcion
<code>instrucciones</code>	Solicitud de ayuda o ejemplos de uso.
<code>anadir_ingredientes</code>	El usuario indica ingredientes disponibles.
<code>recomendar_receta</code>	El usuario solicita una receta con restricciones, tiempo o ingredientes.
<code>sustituir_ingrediente</code>	El usuario pide cambiar un ingrediente por restriccion o preferencia.
<code>ajustar_raciones</code>	El usuario indica un numero de raciones o personas.
<code>lista_compra</code>	El usuario solicita ingredientes faltantes.

El canal textual publica eventos en `event_text.json`. Estos eventos incorporan entidades extraidas, como ingredientes, raciones o restricciones.

```
1 {
2   "source": "text",
3   "intent": "anadir_ingredientes",
4   "timestamp": 1778945740.10,
5   "raw_text": "tengo tomate pan y queso",
6   "entities": {
7     "ingredientes": ["tomate", "pan", "queso"],
8     "ingrediente_principal": "tomate"
9   }
10 }
```

Listing 3: Ejemplo de evento textual publicado por el canal PLN.

[illegible]

Figura 4: Consola de `main_chef_zero_waste.py` clasificando texto y publicando `event_text.json`.

3.2.3. Complementariedad texto-vision

Las intenciones textuales no son una repeticion literal de los gestos. El texto aporta parametros concretos y el gesto aporta la intencion pragmatica o confirmacion visual.

Texto PLN	Gesto	Intencion visual	Complemento
anadir_ingredientes	pizca_sal	iniciar_receta	El texto dice los ingredientes; el gesto indica que se empieza a cocinar.
recomendar_receta	pizca_sal	iniciar_receta	El texto aporta parametros; el gesto confirma la accion.
ajustar_raciones	corte_cuchillo	aumentar_raciones	El texto indica el numero; el gesto expresa partir la receta.
sustituir_ingredientes	sustituir_ingredientes	marcar_sustitucion	Texto indica ingrediente; gesto marca sustitucion.

Ejemplos de fusion no redundante:

- “tengo tomate, pan y queso” + `pizca_sal`: generar una receta con esos ingredientes.

- “divide la receta para 4 personas” + `corte_cuchillo`: adaptar la receta a cuatro raciones.
- “sustituye queso sin lactosa” + gesto de cruz: aplicar sustitucion de ingrediente.

3.2.4. Codigo involucrado en el punto 2

El modulo PLN completo se entrega en estos ficheros:

- `main_chef_zero_waste.py`: entrada textual interactiva.
- `CanalTextoChefZeroWaste.py`: publica `event_text.json`.
- `BowChat.py`, `BowChatChefZeroWaste.py`: vectorizacion Bag of Words y categorias.
- `BowChatChefZeroWaste_E1_Agente.py`: logica del agente ChefZeroWaste.
- `BowChatChefZeroWaste_E2_STM.py`: memoria a corto plazo.
- `Chat.py`, `RobustLinearSVC.py`: infraestructura base del clasificador.

Fragmento clave: el canal textual publica un evento comun que despues consume el integrador.

```
1 save_multimodal_event(  
2     event_path,  
3     "text",  
4     intent,  
5     raw_text=self._last_raw_sentence,  
6     entities=entities,  
7     **extra_event_fields,  
8 )
```

Listing 4: Publicacion del evento textual.

3.3. Fusion semantica tardia

3.3.1. Integrador en Python

El integrador esta implementado en `IntegradorMultimodal.py`. Su funcion es recibir eventos empaquetados desde ambos canales:

- `event_text.json`, producido por `CanalTextoChefZeroWaste.py`.
- `event_vision.json`, producido por `vision/main.py`.

Ambos canales usan `multimodal_utils.py` para escribir eventos JSON de forma atomica y evitar conflictos de lectura/escritura en Windows.

3.3.2. Gestion de asincronia

La fusion no exige que texto y gesto ocurran en el mismo instante. El integrador mantiene una memoria temporal de eventos recientes y compara timestamps. En las pruebas finales se lanza con una ventana de 8000 ms:

```
1 python IntegradorMultimodal.py --window-ms 8000
```

Listing 5: Ejecucion del integrador con ventana temporal.

Conceptualmente, la condicion temporal es:

$$|t_{texto} - t_{vision}| \leq ventana$$

Si una pareja de eventos esta dentro de la ventana, el integrador pasa a validar la compatibilidad semantica.

3.3.3. Validacion semantica

La accion final solo se ejecuta si el par textual-visual se encuentra dentro de la tabla de fusiones validas.

Intencion textual	Intencion visual	Accion final
anadir_ingredientes	iniciar_receta	Recomendar receta usando los ingredientes indicados por texto.
recomendar_receta	iniciar_receta	Confirmar el inicio de la receta solicitada.
ajustar_raciones	aumentar_raciones	Escalar o partir la receta al numero de raciones indicado.
sustituir_ingrediente	marcar_sustitucion	Aplicar la sustitucion indicada por el canal textual.

Si la combinacion no esta en esa tabla, el integrador la rechaza aunque los eventos sean temporalmente cercanos. Por ejemplo, `lista_compra + iniciar_receta` se ignora.

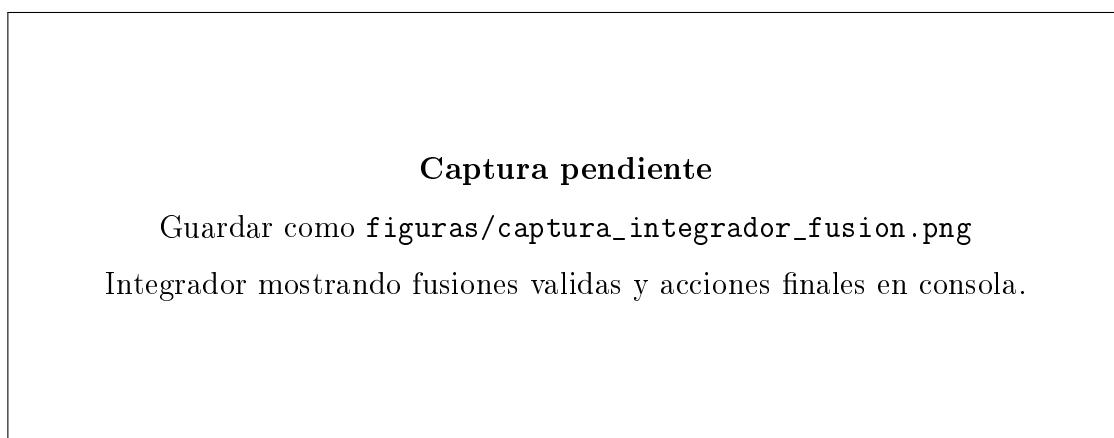


Figura 5: Integrador mostrando fusiones validas y acciones finales en consola.

3.3.4. Evidencia de funcionamiento

En la prueba real de la aplicacion base se observaron fusiones validas como:

```

1 [texto] anadir_ingredientes
2 [vision] iniciar_receta
3 FUSION VALIDA: texto='anadir_ingredientes' + gesto='iniciar_receta'
4 RECETA PROPUESTA: Tostas de tomate aprovechado
5
6 [texto] sustituir_ingrediente
7 [vision] marcar_sustitucion
8 FUSION VALIDA: texto='sustituir_ingrediente' + gesto='marcar_sustitucion'
9 SUSTITUCION: queso -> queso sin lactosa o levadura nutricional

```

Listing 6: Salida esperada de la fusion base.

3.3.5. Codigo involucrado en el punto 3

Ficheros relevantes:

- IntegradorMultimodal.py: memoria temporal, validacion semantica y acciones finales.
- multimodal_utils.py: escritura atomica de eventos JSON.
- lanzar_app_final.ps1: arranque de las tres terminales de la demo obligatoria.

Fragmento clave: el integrador solo ejecuta acciones si la pareja texto-vision esta dentro de la tabla de fusiones validas.

```

1 VALID_FUSIONS = {
2     ("anadir_ingredientes", "iniciar_receta"): "recomendar_receta_con_ingredientes",
3     ("recomendar_receta", "iniciar_receta"): "confirmar_recomendacion_receta",
4     ("ajustar_raciones", "aumentar_raciones"): "partir_en_mas_raciones",
5     ("sustituir_ingrediente", "marcar_sustitucion"): "sustituir_ingrediente",
6 }

```

Listing 7: Tabla de fusiones validas.

4. Descripcion de las ampliaciones

4.1. Ampliacion A: Embeddings multimodales Vision + Texto

4.1.1. Objetivo

La ampliacion opcional implementada sustituye parte de la logica unimodal por un modelo de embeddings multimodales. Se utiliza CLIP, concretamente `openai/clip-vit-base-patch32`, mediante `transformers`. La idea es proyectar comandos textuales libres y recortes visuales de la mano en un espacio semantico compartido, para realizar emparejamiento zero-shot con similitud coseno.

4.1.2. Arquitectura de la ampliacion

En este modo no se ejecutan simultaneamente `vision/main.py` ni `main_chef_zero_waste.py`. El script `clip_zero_shot_multimodal.py` hace de canal multimodal zero-shot y publica eventos compatibles con el integrador existente.

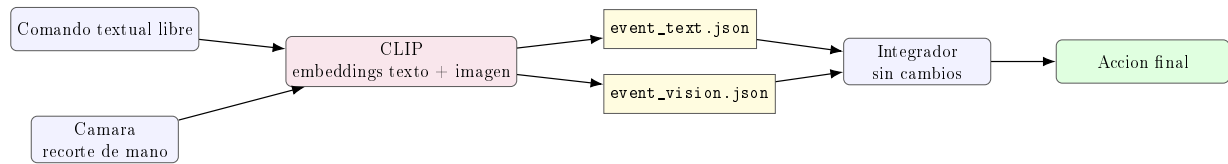


Figura 6: Arquitectura de la ampliación zero-shot con CLIP.

4.1.3. Modelo y espacio compartido

CLIP genera embeddings normalizados para texto e imagen. Dado un embedding visual v y un embedding textual t , la similitud se calcula como:

$$\text{sim}(v, t) = \frac{v \cdot t}{\|v\| \|t\|}$$

En el código, como los embeddings se normalizan previamente, la similitud coseno se obtiene con un producto escalar. El sistema compara:

- El comando textual libre del usuario.
- El recorte de mano detectado por MediaPipe Tasks.
- Prompts descriptivos de los tres gestos obligatorios.

El modelo CLIP se descarga la primera vez y queda cacheado localmente en el directorio del usuario:

```
~/ .cache/huggingface/hub/models-openai-clip-vit-base-patch32
```

Para el recorte de mano se utiliza:

```
models/hand_landmarker.task
```

4.1.4. Gestos obligatorios conservados

La ampliación conserva los tres gestos obligatorios y sus intenciones:

Gesto	Intencion visual	Intencion textual generada
pizca_sal	iniciar_receta	anadir_ingredientes
corte_cuchillo	aumentar_raciones	ajustar_raciones
sustituir_ingrediente	marcar_sustitucion	sustituir_ingrediente

Cuadro 6: Equivalencia semántica usada por CLIP para publicar eventos compatibles con el integrador.

4.1.5. Comandos zero-shot

El usuario puede inventar comandos nuevos en tiempo real. Ejemplos probados:

- quiero empezar a cocinar con tomate pan y queso
- divide la receta para 4 personas
- cambia queso por una alternativa sin lactosa

El sistema extrae entidades simples del texto libre: ingredientes conocidos, numero de raciones y restricciones como `sin_lactosa`. Asi, aunque el texto no pase por el clasificador BoW original, el integrador puede seguir generando una accion final concreta.

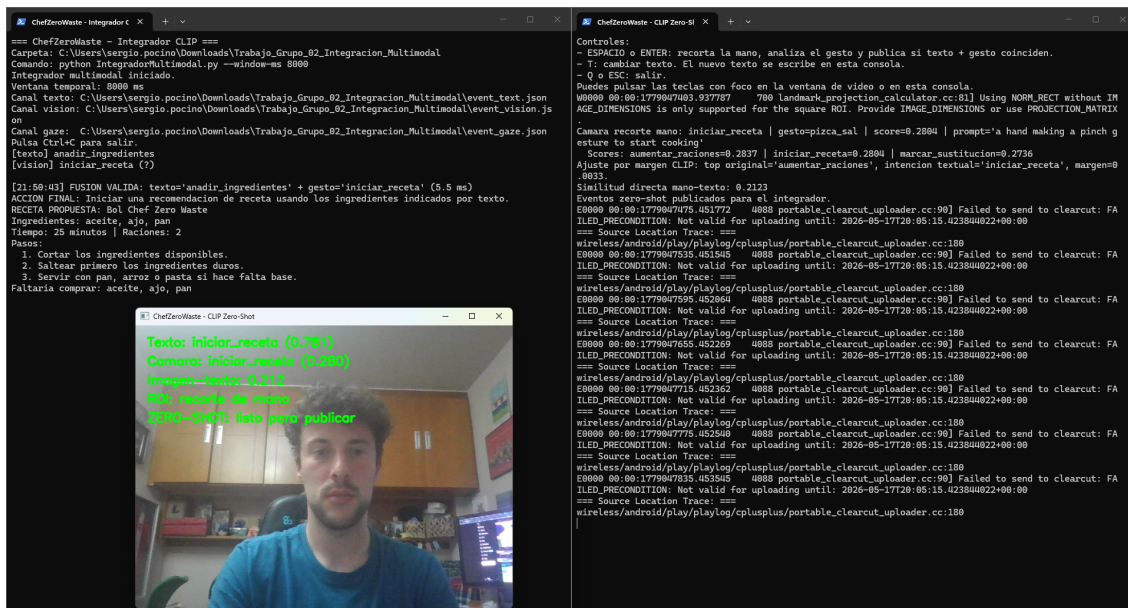


Figura 7: Evidencia de funcionamiento de la ampliacion A con CLIP.

4.1.6. Evidencia de funcionamiento de la ampliacion

La prueba final de la ampliacion produjo las tres fusiones esperadas:

```

1 Texto: iniciar_receta | gesto=pizca_sal | score=0.8551
2 Camara recorte mano: iniciar_receta | gesto=pizca_sal | score=0.2541
3 Eventos zero-shot publicados para el integrador.
4
5 Texto: aumentar_raciones | gesto=corte_cuchillo | score=0.8742
6 Camara recorte mano: aumentar_raciones | gesto=corte_cuchillo | score=0.2903
7 Eventos zero-shot publicados para el integrador.
8
9 Texto: marcar_sustitucion | gesto=sustituir_ingredientes | score=0.8606
10 Camara recorte mano: marcar_sustitucion | gesto=sustituir_ingredientes | score=0.2985
11 Eventos zero-shot publicados para el integrador.
```

Listing 8: Logs resumidos de la ampliacion CLIP.

El integrador recibio esos eventos y ejecuto las acciones finales:

```

1 FUSION VALIDA: texto='anadir_ingredientes' + gesto='iniciar_receta'
2 RECETA PROPUESTA: Tostas de tomate aprovechado
3
4 FUSION VALIDA: texto='ajustar_raciones' + gesto='aumentar_raciones'
5 RACIONES: Dividir o escalar la receta para 4 raciones.
6
```

```

7 FUSION VALIDA: texto='sustituir_ingredient' + gesto='marcar_sustitucion'
8 SUSTITUCION: queso -> queso sin lactosa o levadura nutricional (sin_lactosa)

```

Listing 9: Logs resumidos del integrador usando eventos publicados por CLIP.

4.1.7. Ejecucion de la ampliacion

La ampliacion se lanza con:

```

1 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_clip.ps1

```

Listing 10: Lanzamiento de la ampliacion CLIP.

El lanzador abre dos terminales:

- Integrador con `python IntegradorMultimodal.py -window-ms 8000`.
- Demo zero-shot con `python clip_zero_shot_multimodal.py`.

No debe ejecutarse `vision/main.py` al mismo tiempo, porque ambos modos usan la webcam.

4.1.8. Codigo involucrado en la ampliacion

Ficheros relevantes:

- `clip_zero_shot_multimodal.py`: carga CLIP, recorta la mano, calcula similitudes y publica eventos.
- `lanzar_ampliacion_clip.ps1`: arranque del integrador y demo CLIP.
- `models/hand_landmarker.task`: detector de mano usado para recortar la region visual.
- `IntegradorMultimodal.py`: se reutiliza sin cambiar su contrato de entrada.

Fragmento clave: texto e imagen se normalizan y se comparan con producto escalar, equivalente a similitud coseno.

```

1 image_embedding = normalize(model.get_image_features(**image_inputs))
2 text_embedding = normalize(model.get_text_features(**text_inputs))
3 similarity = float(np.dot(image_embedding, text_embedding))

```

Listing 11: Comparacion CLIP en espacio compartido.

4.2. Ampliacion B: Integracion de voz en tiempo real

4.2.1. Objetivo

La ampliacion B elimina la introduccion manual de texto por teclado. En vez de escribir en `main_chef_zero_waste.py`, el usuario habla por el microfono. El audio real se captura, se transcribe con un motor ASR local y la frase resultante se introduce en el mismo modulo PLN de ChefZeroWaste.

4.2.2. Arquitectura de la ampliacion

Se ha anadido el script:

`voz_asr_chef_zero_waste.py`

Este canal usa `sounddevice` para capturar audio y `faster-whisper` (modelo Systran/`faster-whisper-medium`) para transcribir la frase. Luego reutiliza `CanalTextoChefZeroWaste.py` para clasificar, extraer entidades y publicar el evento.

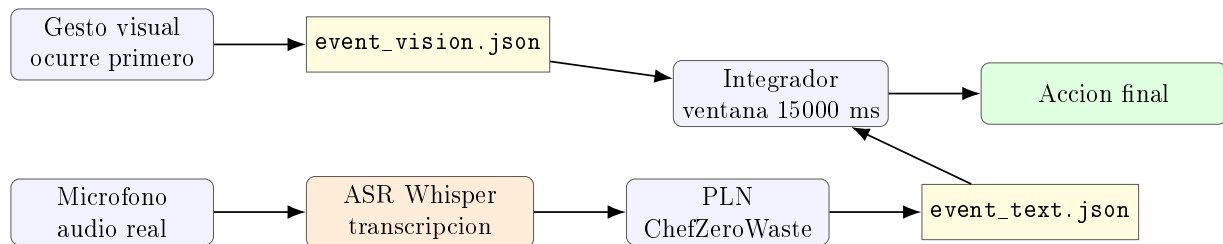


Figura 8: Arquitectura de la ampliacion de voz en tiempo real.

4.2.3. Captura de audio y transcripcion

El canal de voz implementa una deteccion simple de actividad de voz:

1. Calibra el ruido ambiente.
2. Calcula energia RMS por bloques de audio.
3. Empieza a grabar cuando la energia supera el umbral de inicio.
4. Termina la frase cuando detecta silencio durante un tiempo suficiente.
5. Transcribe el audio con Whisper.
6. Clasifica la frase transcrita con el PLN.

El evento textual publicado conserva el contrato del integrador y anade metadatos ASR:

```

1 {
2   "source": "text",
3   "intent": "anadir_ingredientes",
4   "raw_text": "tengo tomate pan y queso",
5   "entities": {
6     "ingredientes": ["tomate", "pan", "queso"],
7     "ingrediente_principal": "tomate"
8   },
9   "asr": true,
10  "asr_engine": "faster-whisper",
11  "asr_language": "es"
12 }
  
```

Listing 12: Ejemplo de evento textual generado por voz.

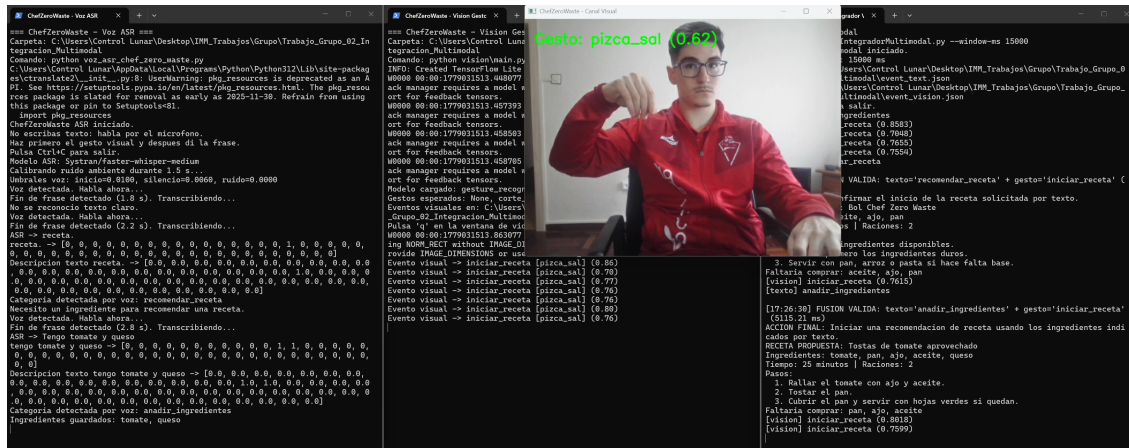


Figura 9: Canal de voz detectando frase, transcribiendo con Whisper y publicando evento textual.

4.2.4. Reto de asincronia real

El requisito especifica que en comunicacion humana natural el gesto suele preceder a la voz. Para demostrarlo, el lanzador de esta ampliacion ejecuta el integrador con una ventana temporal mas amplia:

```
1 python IntegradorMultimodal.py --window-ms 15000
```

Listing 13: Integrador con memoria temporal para la ampliacion de voz.

La secuencia esperada es:

1. El usuario hace el gesto, por ejemplo `pizca_sal`.
2. El canal visual publica `iniciar_receta`.
3. El usuario dice una frase, por ejemplo “tengo tomate pan y queso”.
4. El canal ASR espera al fin de la frase, transcribe y clasifica `anadir_ingredientes`.
5. El integrador todavia conserva el gesto en memoria y fusiona ambos eventos.

Esto demuestra que el sistema no depende de simultaneidad exacta: el gesto puede quedar guardado mientras termina el procesamiento de voz.

4.2.5. Ejecucion

La ampliacion se lanza con:

```
1 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_voz.ps1
```

Listing 14: Lanzamiento de la ampliacion de voz.

El lanzador abre:

- Integrador con ventana de 15000 ms.
- Canal visual de gestos.
- Canal de voz ASR.

Comandos auxiliares:

```
1 python voz_asr_chef_zero_waste.py --list-devices
2 python voz_asr_chef_zero_waste.py --demo-text "tengo tomate pan y queso"
```

Listing 15: Comandos auxiliares del canal ASR.

4.2.6. Codigo involucrado en la ampliacion

Ficheros relevantes:

- `voz_asr_chef_zero_waste.py`: captura microfono, detecta voz/silencio, transcribe y llama al PLN.
- `lanzar_ampliacion_voz.ps1`: arranca integrador, vision y ASR.
- `CanalTextoChefZeroWaste.py`: reutilizado para clasificar la transcripcion y publicar `event_text.json`.
- `IntegradorMultimodal.py`: mantiene el gesto en memoria hasta que llega la transcripcion.

Fragmento clave: el canal ASR espera silencio final antes de transcribir y publicar el texto.

```
1 if duration_seconds >= min_phrase_seconds and silence_seconds >= end_silence_seconds:
2     audio = np.concatenate(chunks)
3     text, metadata = transcribe_audio(whisper, audio, language="es")
4     processor.process_sentence(text, metadata)
```

Listing 16: Final de frase por silencio en el canal ASR.

4.3. Ampliacion C: Logica de desambiguacion mutua

4.3.1. Objetivo

La ampliacion C implementa el principio de desambiguacion mutua en el integrador central. La idea es que cuando un canal (texto o vision) tiene un nivel de confianza muy bajo debido a ruido, palabras ambiguas o un gesto parcialmente fuera de cuadro, el otro canal con alta certeza pueda corregir la intencion erronea y ejecutar la accion correcta.

4.3.2. Arquitectura de la desambiguacion

Esta logica se ha implementado en el metodo `check_fusion()` de la clase `MultimodalIntegrator` en `IntegradorMultimodal.py`. El flujo completo es:

1. Se intenta la fusion directa: buscar (`intencion_textual`, `intencion_visual`) en `VALID_FUSIONS`.
2. Si la fusion directa falla, invoca `_try_disambiguate(text_event, visual_event)`.
3. La desambiguacion evalua dos casos complementarios:
 - **Caso 1**: confianza textual menor que `LOW_TEXT_CONFIDENCE` y visual mayor o igual a `HIGH_VISUAL_CONFIDENCE`, el gesto corrige al texto.

- **Caso 2:** confianza visual menor que `LOW_VISUAL_CONFIDENCE` y textual mayor o igual a `HIGH_TEXT_CONFIDENCE`, el texto corrige al gesto.
4. Si ninguno de los dos casos aplica (por ejemplo, ambos canales tienen baja confianza), la fusion se rechaza como en la version base.

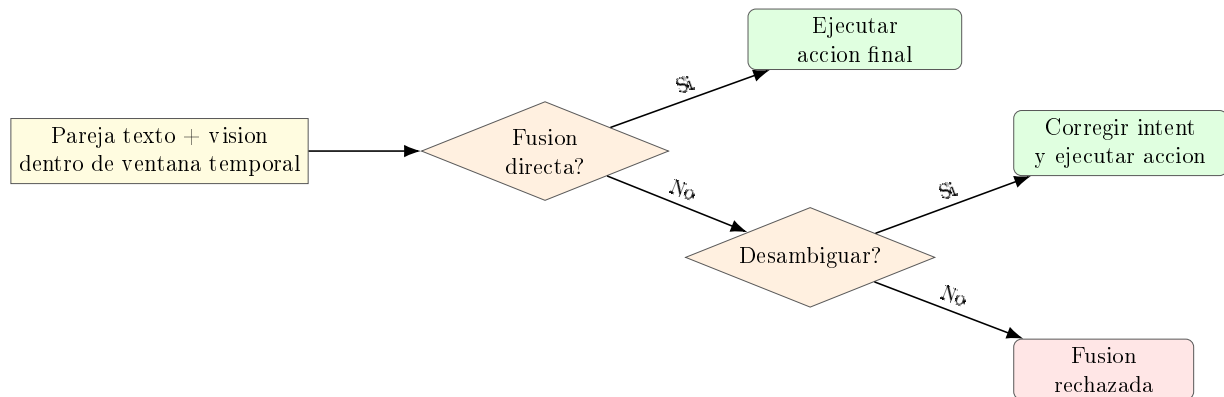


Figura 10: Flujo de decision del integrador con desambiguacion mutua.

4.3.3. Umbrales de confianza

Los umbrales que controlan la desambiguacion estan definidos como constantes del modulo:

Parametro	Valor	Significado
LOW_TEXT_CONFIDENCE	0.45	Confianza textual insuficiente.
HIGH_TEXT_CONFIDENCE	0.78	Confianza textual fiable para corregir.
LOW_VISUAL_CONFIDENCE	0.55	Confianza visual insuficiente.
HIGH_VISUAL_CONFIDENCE	0.78	Confianza visual fiable para corregir.

Cuadro 7: Umbrales de confianza para la desambiguacion mutua.

4.3.4. Tablas de correccion mutua

Para poder corregir la intencion del canal debil, el integrador necesita saber que intencion esperar del canal corregido. Esto se define en dos tablas:

Mapa de intencion visual a posibles intenciones textuales (`VISUAL_TO_TEXT_INTENT`):

Intencion visual	Intenciones textuales candidatas
iniciar_receta	recomendar_receta, anadir_ingredientes
aumentar_raciones	ajustar_raciones
marcar_sustitucion	sustituir_ingrediente

Cuadro 8: Mapa visual a texto para desambiguacion (gesto corrige texto).

Mapa de intencion textual a intencion visual (`TEXT_TO_VISUAL_INTENT`):

Intencion textual	Intencion visual esperada
anadir_ingredientes	iniciar_receta
recomendar_receta	iniciar_receta
ajustar_raciones	aumentar_raciones
sustituir_ingrediente	marcar_sustitucion

Cuadro 9: Mapa texto a visual para desambiguacion (texto corrige gesto).

4.3.5. Origen de la confianza en cada canal

El **canal textual** (CanalTextoChefZeroWaste.py) calcula la confianza combinando dos fuentes:

- **Reglas textuales:** si una unica regla coincide con la frase, la confianza es 0.92. Si varias reglas coinciden (frase ambigua), la confianza cae a 0.34.
- **Clasificador BoW:** se aplica `decision_function` sobre el vector Bag of Words y se normaliza con softmax para obtener una probabilidad.
- **Combinacion:** si regla y clasificador coinciden, la confianza sube; si discrepan, se limita.

La confianza se publica dentro del campo `confidence` de `event_text.json`.

El **canal visual** (`vision/main.py`) obtiene la confianza directamente del score del modelo MediaPipe y la publica en `event_vision.json`.

4.3.6. Evidencia de funcionamiento

La demo incluye tres escenarios de desambiguacion que se ejecutan con:

```
1 python IntegradorMultimodal.py --demo
```

Listing 17: Demo de desambiguacion mutua.

Escenario 1 – Texto ambiguo corregido por gesto claro. El PLN recibe una frase ruidosa que activa varias reglas y clasifica como `lista_compra` con confianza 0.30. El gesto `pizca_sal` se detecta con confianza 0.92. El integrador infiere que la intencion textual real era `recomendar_receta`.

Escenario 2 – Gesto ambiguo corregido por texto claro. El modelo visual detecta `aumentar_raciones` con confianza 0.40 porque la mano estaba parcialmente fuera de cuadro. El texto dice `sustituir_ingrediente` con confianza 0.95. El integrador corrige el gesto a `marcar_sustitucion`.

Escenario 3 – Ambos canales con baja confianza. Ni el texto (0.25) ni la vision (0.35) son fiables. El sistema rechaza la fusion sin inventar una correccion.

```
1 --- Escenario 1: Texto ambiguo, gesto claro ---
2 DESAMBIGUACION MUTUA: canal 'text' corregido por canal 'vision'
3   Intent original del text: 'lista_compra' (confianza 0.30)
4   Corregido a 'recomendar_receta' gracias a 'iniciar_receta'
5   del canal vision (confianza 0.92)
6 FUSION VALIDA: texto='recomendar_receta' + gesto='iniciar_receta'
7
8 --- Escenario 2: Gesto ambiguo, texto claro ---
9 DESAMBIGUACION MUTUA: canal 'vision' corregido por canal 'text'
10  Intent original del vision: 'aumentar_raciones' (confianza 0.40)
11  Corregido a 'marcar_sustitucion' gracias a
```

```

12 'sustituir_ingrediente' del canal text (confianza 0.95)
13 FUSION VALIDA: texto='sustituir_ingrediente' + gesto='marcar_sustitucion'
14 SUSTITUCION: leche -> bebida vegetal (sin_lactosa)
15
16 --- Escenario 3: Ambos canales con baja confianza ---
17 Fusion ignorada: texto='lista_compra' + gesto='aumentar_raciones'
18 no es una combinacion valida.

```

Listing 18: Salida de los escenarios de desambiguacion mutua.

En la prueba real con webcam y chat PLN se verifico el escenario 1: al escribir una frase ambigua como “quiero algo de cocina receta compra lista con tomate” (confianza 0.34) y hacer el gesto pizca_sal con confianza alta (0.89), el integrador corrigio la intencion textual y ejecuto la receta correcta.

```

1 [vision] iniciar_receta (0.8278)
2 [vision] iniciar_receta (0.891)
3 [texto] lista_compra
4
5 DESAMBIGUACION MUTUA: canal 'text' corregido por canal 'vision'
6 Intent original del text: 'lista_compra' (confianza 0.34)
7 Corregido a 'recomendar_receta' gracias a 'iniciar_receta'
8 del canal vision (confianza 0.89)
9
10 FUSION VALIDA: texto='recomendar_receta' + gesto='iniciar_receta'
11 ACCION FINAL: Confirmar el inicio de la receta solicitada.
12 RECETA PROPUESTA: Tostas de tomate aprovechado

```

Listing 19: Log real de desambiguacion en vivo con webcam y chat.

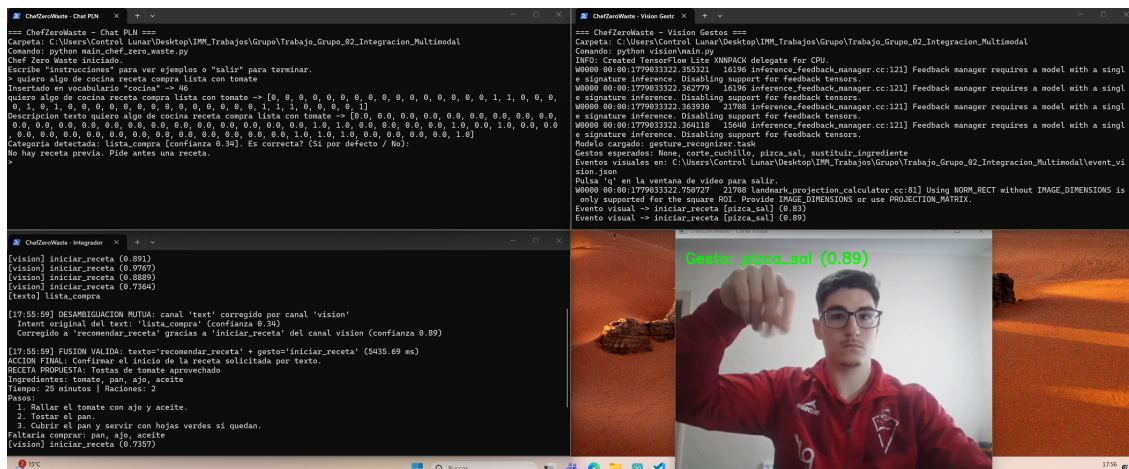


Figura 11: Integrador mostrando la desambiguacion mutua en vivo: el gesto claro corrige la frase ambigua del canal textual.

4.3.7. Salida JSON con metadatos de desambiguacion

Cuando se aplica desambiguacion, el JSON de fusion escrito en `event_fusion.json` incluye un campo adicional `disambiguation` que permite trazar la correccion:

```

1 {
2   "source": "fusion",
3   "action": "sustituir_ingrediente",
4   "text_intent": "sustituir_ingrediente",
5   "visual_intent": "marcar_sustitucion",
6   "disambiguation": {
7     "disambiguation": true,
8     "corrected_channel": "vision",
9     "original_intent": "aumentar_raciones",
10    "corrected_intent": "marcar_sustitucion",

```

```

11     "corrected_by_channel": "text",
12     "corrected_by_intent": "sustituir_ingrediente",
13     "low_confidence": 0.40,
14     "high_confidence": 0.95
15 }
16 }

```

Listing 20: Evento de fusion con metadatos de desambiguacion.

4.3.8. Codigo involucrado en la ampliacion

La desambiguacion no requiere ficheros nuevos. Se implementa dentro de los ya existentes:

- `IntegradorMultimodal.py`: metodo `_try_disambiguate()`, umbrales de confianza, tablas `VISUAL_TO_TEXT_INTENT` y `TEXT_TO_VISUAL_INTENT`, y escenarios de demo.
- `CanalTextoChefZeroWaste.py`: metodo `classify_intent()` que produce la confianza textual, combinando reglas y clasificador BoW.
- `vision/main.py`: publica `confidence` en el evento visual desde el score del modelo MediaPipe.

Fragmento clave: la logica de desambiguacion dentro del integrador.

```

1  # Caso 1: texto ambiguo, vision segura
2  if text_conf < LOW_TEXT_CONFIDENCE and visual_conf >= HIGH_VISUAL_CONFIDENCE:
3      candidates = VISUAL_TO_TEXT_INTENT.get(visual_event.intent, [])
4      for corrected_text_intent in candidates:
5          rule = VALID_FUSIONS.get((corrected_text_intent, visual_event.intent))
6          if rule is not None:
7              # Corregir intent textual y ejecutar fusion
8              ...
9
10 # Caso 2: gesto ambiguo, texto seguro
11 if visual_conf < LOW_VISUAL_CONFIDENCE and text_conf >= HIGH_TEXT_CONFIDENCE:
12     corrected_visual_intent = TEXT_TO_VISUAL_INTENT.get(text_event.intent)
13     if corrected_visual_intent is not None:
14         rule = VALID_FUSIONS.get((text_event.intent, corrected_visual_intent))
15         if rule is not None:
16             # Corregir intent visual y ejecutar fusion
17             ...

```

Listing 21: Logica de desambiguacion mutua en el integrador.

4.4. Ampliacion D: Head Tracking como tercer canal de percepcion pasiva

4.4.1. Objetivo

La ampliacion D integra un tercer canal de percepcion pasiva que utiliza los landmarks de la cara (MediaPipe Face Landmarker, 478 puntos) para calcular la orientacion de la cabeza con `cv2.solvePnP`. Este canal aporta contexto espacial: a donde mira el usuario en la pantalla, complementando el que (PLN) y el como (gesto).

4.4.2. Arquitectura de la ampliacion

El script `vision/gaze_head_tracking.py` ejecuta en un mismo bucle de frames:

1. MediaPipe Face Landmarker para detectar 478 landmarks faciales.
2. Seleccin de 6 landmarks clave (nariz, menton, esquinas de ojos y boca).
3. `cv2.solvePnP` con un modelo facial 3D generico para obtener los angulos de Euler (yaw, pitch, roll).
4. Clasificacion de la zona de la pantalla segun los angulos.
5. MediaPipe Gesture Recognizer para detectar gestos de mano.
6. Publicacion de `event_gaze.json` y `event_vision.json`.

Al combinar ambos modelos en un solo proceso, se evita el conflicto de camara que surgiria si se ejecutasen dos scripts con acceso simultaneo a la webcam.

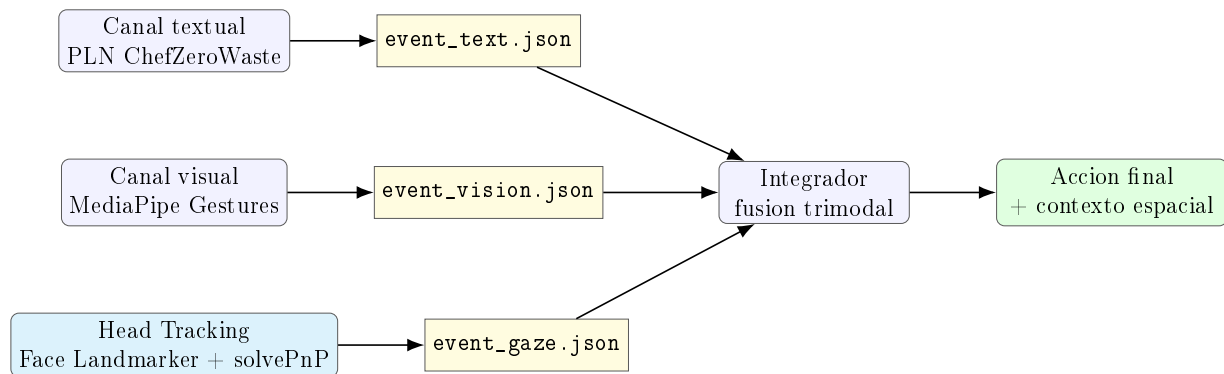


Figura 12: Arquitectura con tres canales: texto, gestos y head tracking.

4.4.3. Zonas de la pantalla

La pantalla se divide en zonas logicas que representan secciones de la interfaz de cocina:

Zona	Condicion	Intencion gaze	Seccion
Izquierda	$\text{yaw} < -12^\circ$	<code>mirada_ingredientes</code>	ingredientes
Derecha	$\text{yaw} > 12^\circ$	<code>mirada_pasos</code>	pasos
Arriba	$\text{pitch} < -10^\circ$	<code>mirada_titulo</code>	titulo
Centro	resto	<code>mirada_receta</code>	receta

Cuadro 10: Mapeo de orientacion de cabeza a zonas de la interfaz.

4.4.4. Estimacion de la pose con solvePnP

Se seleccionan 6 landmarks faciales (indices 1, 152, 33, 263, 61, 291 del Face Landmarker) que corresponden a la punta de la nariz, el menton, las esquinas de los ojos y las esquinas de la boca. Estos puntos 2D se emparejan con un modelo 3D generico del rostro humano (en milimetros) y se pasan a `cv2.solvePnP` para obtener los vectores de rotacion y traslacion. Los angulos de Euler se extraen con `cv2.decomposeProjectionMatrix`.

```

1  FACE_3D_MODEL = np.array([
2      [0.0, 0.0, 0.0],           # Nariz
3      [0.0, -63.6, -12.5],      # Menton
4      [-43.3, 32.7, -26.0],     # Ojo izquierdo
5      [43.3, 32.7, -26.0],      # Ojo derecho
6      [-28.9, -28.9, -24.1],    # Boca izquierda
7      [28.9, -28.9, -24.1],     # Boca derecha
8  ], dtype=np.float64)
9
10 success, rotation_vec, translation_vec = cv2.solvePnP(
11     FACE_3D_MODEL, points_2d, camera_matrix, dist_coeffs,
12     flags=cv2.SOLVEPNP_ITERATIVE
13 )
14 rotation_matrix, _ = cv2.Rodrigues(rotation_vec)
15 projection = np.hstack((rotation_matrix, translation_vec))
16 _, _, _, _, _, _, euler = cv2.decomposeProjectionMatrix(projection)
17 yaw, pitch, roll = euler[1], euler[0], euler[2]

```

Listing 22: Estimacion de yaw, pitch y roll con solvePnP.

4.4.5. Canal gaze pasivo

El canal gaze es aditivo y no bloquea la fusion texto+gesto. Cuando una fusion bimodal se produce con exito, el integrador busca si hay un evento de mirada reciente dentro de la ventana temporal. Si existe, lo anota como contexto espacial en el resultado de fusion. Si no existe, la fusion procede normalmente sin contexto de mirada.

Formato del evento gaze:

```

1  {
2      "source": "gaze",
3      "intent": "mirada_ingredientes",
4      "timestamp": 1779033322.75,
5      "zone": "izquierda",
6      "section": "ingredientes",
7      "yaw": -22.5,
8      "pitch": 3.0,
9      "confidence": 0.68
10 }

```

Listing 23: Ejemplo de evento gaze publicado por el head tracker.

Resultado de fusion enriquecido con gaze:

```

1  {
2      "source": "fusion",
3      "action": "recomendar_receta_con_ingredientes",
4      "text_intent": "anadir_ingredientes",
5      "visual_intent": "iniciar_receta",
6      "gaze_context": {
7          "gaze_intent": "mirada_ingredientes",
8          "zone": "izquierda",
9          "section": "ingredientes",
10         "yaw": -22.5,
11         "pitch": 3.0,
12         "confidence": 0.68
13     }
14 }

```

Listing 24: Fusion trimodal con contexto de mirada.

4.4.6. Evidencia de funcionamiento

La demo del integrador incluye tres escenarios de head tracking:


```
1 python IntegradorMultimodal.py --demo
```

Listing 25: Demo de head tracking.

Escenario D1. Usuario dice ingredientes (texto), hace pizca_sal (gesto) y mira a la izquierda (gaze). La fusion ejecuta receta indicando foco de atencion en “ingredientes”.

Escenario D2 – Fusion bimodal sin gaze. El usuario no mira a ninguna zona especifica. La fusion texto+gesto funciona igual que antes, sin campo gaze_context.

Escenario D3 – Sustitucion con mirada a pasos. El usuario sustituye un ingrediente mientras mira a la seccion “pasos”.

```
1 --- Escenario D1: Fusion trimodal (texto + gesto + mirada) ---
2 FUSION VALIDA: texto='anadir_ingredientes' + gesto='iniciar_receta'
3 ACCION FINAL: Iniciar receta con ingredientes indicados.
4 FOCO DE ATENCION: seccion 'ingredientes'
5 (zona izquierda, yaw=-22.5, pitch=3.0)
6
7 --- Escenario D2: Fusion bimodal sin gaze (compatibilidad) ---
8 FUSION VALIDA: texto='ajustar_raciones' + gesto='aumentar_raciones'
9 ACCION FINAL: Ajustar la receta a 6 raciones.
10
11 --- Escenario D3: Sustitucion con mirada a 'pasos' ---
12 FUSION VALIDA: texto='sustituir_ingredientes' + gesto='marcar_sustitucion'
13 ACCION FINAL: Aplicar sustitucion del ingrediente.
14 FOCO DE ATENCION: seccion 'pasos'
15 (zona derecha, yaw=19.0, pitch=1.5)
```

Listing 26: Salida de los escenarios de head tracking.

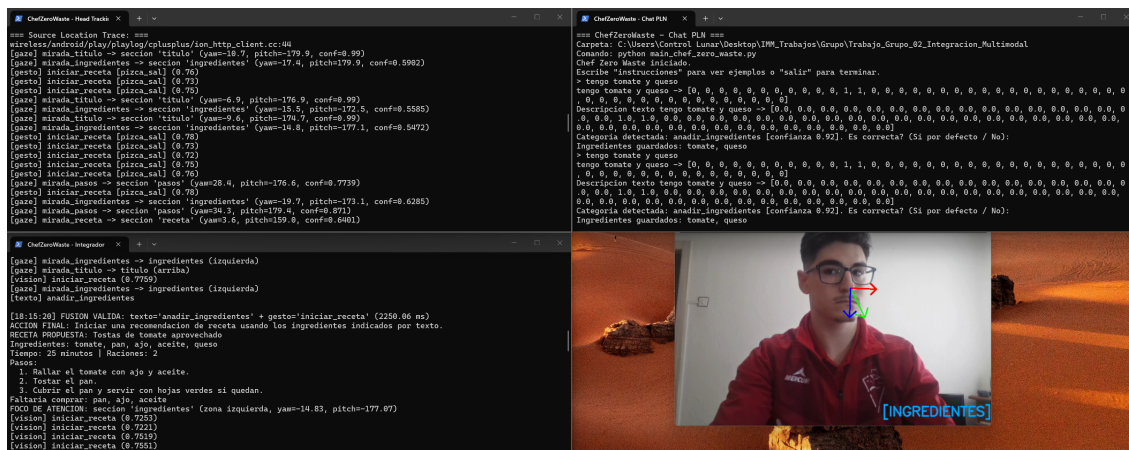


Figura 13: Head tracking mostrando los ejes de orientacion de la cabeza, la zona detectada y el gesto reconocido simultaneamente.

4.4.7. Ejecucion

La ampliacion se lanza con:

```
1 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_gaze.ps1
```

Listing 27: Lanzamiento de la ampliacion de head tracking.

El lanzador abre tres terminales:

- Integrador con ventana de 8000 ms.
- Head tracking + gestos combinados (vision/gaze_head_tracking.py).
- Canal textual ChefZeroWaste.

4.4.8. Codigo involucrado en la ampliacion

Ficheros relevantes:

- vision/gaze_head_tracking.py: Face Landmarker + solvePnP + Gesture Recognizer en un solo bucle.
- models/face_landmarker.task: modelo Face Landmarker de MediaPipe.
- lanzar_ampliacion_gaze.ps1: arranque de integrador, head tracking y PLN.
- IntegradorMultimodal.py: incluye _find_gaze_context() y lee event_gaze.json.

Fragmento clave: el integrador busca contexto gaze cuando se produce una fusion texto+gesto.

```
1 def _find_gaze_context(self, text_event, visual_event):
2     if not self.gaze_events:
3         return None
4     fusion_center = min(text_event.timestamp, visual_event.timestamp)
5     best_gaze = None
6     best_delta = None
7     for gaze_event in self.gaze_events:
8         delta = abs((gaze_event.timestamp - fusion_center).total_seconds())
9         if delta <= self.time_window.total_seconds():
10             if best_delta is None or delta < best_delta:
11                 best_gaze = gaze_event
12                 best_delta = delta
13     if best_gaze is None:
14         return None
15     return {
16         "zone": best_gaze.payload.get("zone"),
17         "section": best_gaze.payload.get("section"),
18         "yaw": best_gaze.payload.get("yaw"),
19         ...
20     }
```

Listing 28: Busqueda de contexto gaze en el integrador.

5. Instrucciones de ejecucion

5.1. Aplicacion obligatoria

```
1 cd Trabajo_Grupo_02_Integracion_Multimodal
2 powershell -ExecutionPolicy Bypass -File .\lanzar_app_final.ps1
```

Listing 29: Lanzar la aplicacion final obligatoria.

Este lanzador abre tres terminales:

- Integrador.
- Canal visual de gestos.
- Canal textual ChefZeroWaste.

5.2. Ampliacion CLIP

```
1 cd Trabajo_Grupo_02_Integracion_Multimodal
2 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_clip.ps1
```

Listing 30: Lanzar la ampliacion de embeddings multimodales.

Controles del modo CLIP:

- ESPACIO o ENTER: analiza el recorte de mano y publica si coincide con el texto.
- T: cambia el comando textual.
- Q o ESC: sale del modo CLIP.

5.3. Ampliacion voz ASR

```
1 cd Trabajo_Grupo_02_Integracion_Multimodal
2 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_voz.ps1
```

Listing 31: Lanzar la ampliacion de voz en tiempo real.

En esta demo se recomienda hacer primero el gesto y despues hablar. El integrador mantiene el gesto en memoria durante 15000 ms para esperar a que el canal ASR termine de transcribir.

5.4. Ampliacion Head Tracking

```
1 cd Trabajo_Grupo_02_Integracion_Multimodal
2 powershell -ExecutionPolicy Bypass -File .\lanzar_ampliacion_gaze.ps1
```

Listing 32: Lanzar la ampliacion de head tracking + gestos.

Esta demo abre tres terminales: integrador, head tracking con gestos combinados, y chat PLN. El orden de prueba es:

1. Girar la cabeza hacia una seccion (izquierda=ingredientes, derecha=pasos).
2. Hacer un gesto con la mano.
3. Escribir la frase en el chat PLN.
4. Verificar que el integrador muestra FOCO DE ATENCION junto con la fusion.